DS HW 8

1. Simulate BFS and DFS traversal for the given graph

Starting vertex: A. Adjacent vertices are visited in alphabetical order.

```
#include <stdio.h>
#define N 7

/* Undirected graph read from the first diagram:
   A-B, A-D, A-G, B-E, B-F, E-G, F-C, F-D */
int graph[N][N] = {
    /* A */ {0,1,0,1,0,0,1},
    /* B */ {1,0,0,0,1,1,0},
    /* C */ {0,0,0,0,0,1,0},
    /* D */ {1,0,0,0,0,1,0},
    /* E */ {0,1,0,0,0,0,1},
    /* F */ {0,1,1,1,0,0,0},
    /* G */ {1,0,0,0,1,0,0}
};
int visited[N];

void bfs(int start) {
    int q[N], front=0, rear=0, i, v;
    for (i=0; i<N; i++) visited[i]=0;
    q[rear++]=start; visited[start]=1;
    printf("BFS: ");
    while (front<rear) {
        v=q[front++];
        printf("%c ", 'A'+v);
        for (i=0; i<N; i++)
            if (graph[v][i] && !visited[i]) {
                visited[i]=1; q[rear++]=i;
            }
    }
    printf("\n");
}
void dfs(int v) {
    int i;
    visited[v]=1;
    printf("%c ", 'A'+v);
    for (i=0; i<N; i++)
        if (graph[v][i] && !visited[i]) dfs(i);
}
int main(void) {
    int i;
    bfs(0); /* start at A */
    for (i=0; i<N; i++) visited[i]=0;
    printf("DFS: "); dfs(0); printf("\n");
    return 0;
}
```

Output:

```
BFS: A B D G E F C
DFS: A B E G F C D
```

2. Find the Minimum Spanning Tree using Prim's and Kruskal's algorithms

Edges: A-B=1, A-C=7, B-C=5, B-D=4, B-E=3, C-E=6, D-E=2.

```
#include <stdio.h>
#define V 5
#define E 7
#define INF 999

typedef struct { int u, v, w; } Edge;
Edge edges[E] = {
    {0,1,1}, {0,2,7}, {1,2,5}, {1,3,4},
    {1,4,3}, {2,4,6}, {3,4,2}
};
int parent[V];

int find(int x) {
    while (parent[x] != x) {
        parent[x] = parent[parent[x]];
        x = parent[x];
    }
    return x;
}
void kruskal(void) {
    int i,j,total=0,count=0;
    Edge t;
```

```
    for(i=0;i<V;i++) parent[i]=i;
    for(i=0;i<E-1;i++)
        for(j=0;j<E-i-1;j++)
            if(edges[j].w>edges[j+1].w) {
                t=edges[j]; edges[j]=edges[j+1]; edges[j+1]=t;
            }
    printf("Kruskal MST:\n");
    for(i=0;i<E && count<V-1;i++) {
        int a=find(edges[i].u), b=find(edges[i].v);
        if(a!=b) {
            printf("%c-%c : %d\n",'A'+edges[i].u,'A'+edges[i].v,edges[i].w);
            parent[a]=b; total+=edges[i].w; count++;
        }
    }
    printf("Total cost = %d\n",total);
}
void prim(void) {
    int cost[V][V] = {
        {0,1,7,INF,INF},
        {1,0,5,4,3},
        {7,5,0,INF,6},
        {INF,4,INF,0,2},
        {INF,3,6,2,0}
    };
    int used[V]={0}, i,j,k,u=0,v=0,min,total=0;
    used[0]=1;
    printf("Prim MST:\n");
    for(i=0;i<V-1;i++) {
        min=INF;
        for(j=0;j<V;j++) if(used[j])
            for(k=0;k<V;k++) if(!used[k] && cost[j][k]<min) {
                min=cost[j][k]; u=j; v=k;
            }
        printf("%c-%c : %d\n",'A'+u,'A'+v,min);
        used[v]=1; total+=min;
    }
    printf("Total cost = %d\n",total);
}
int main(void) { prim(); kruskal(); return 0; }
```

Output:

```
Prim MST:
A-B : 1
B-E : 3
E-D : 2
B-C : 5
Total cost = 11

Kruskal MST:
A-B : 1
D-E : 2
B-E : 3
B-C : 5
Total cost = 11
```

3. Find the Minimum Spanning Tree for the given graph

Enter the number of vertices and the adjacency matrix.

```
/* General Prim's algorithm for an undirected weighted graph.
   Enter the number of vertices, then the adjacency matrix.
   Use 0 for no edge (except the diagonal). */
#include <stdio.h>
#define MAX 20
#define INF 999999

int main(void) {
    int n, a[MAX][MAX], used[MAX]={0};
    int i,j,k,u=0,v=0,min,total=0;
    printf("Number of vertices: ");
    scanf("%d",&n);
    printf("Enter adjacency matrix (0 means no edge):\n");
    for(i=0;i<n;i++) for(j=0;j<n;j++) scanf("%d",&a[i][j]);
    used[0]=1;
    printf("MST edges:\n");
    for(k=0;k<n-1;k++) {
        min=INF;
        for(i=0;i<n;i++) if(used[i])
            for(j=0;j<n;j++)
                if(!used[j] && a[i][j]>0 && a[i][j]<min) {
                    min=a[i][j]; u=i; v=j;
                }
        if(min==INF) { printf("Graph is disconnected.\n"); return 0; }
```

```
        printf("%d - %d : %d\n",u,v,min);
        used[v]=1; total+=min;
    }
    printf("Minimum cost = %d\n",total);
    return 0;
}
```

Output:

```
Sample run (example graph):
Number of vertices: 4
Enter adjacency matrix:
0 1 3 0
1 0 2 4
3 2 0 5
0 4 5 0
MST edges:
0 - 1 : 1
1 - 2 : 2
1 - 3 : 4
Minimum cost = 7
```

4. Dijkstra's algorithm

Enter the number of vertices, adjacency matrix, and source vertex.

```
/* Dijkstra's shortest-path algorithm for a non-negative,
   undirected weighted graph represented by an adjacency matrix.
   Enter 0 where there is no edge. */
#include <stdio.h>
#define MAX 20
#define INF 999999

int main(void) {
    int n, src, a[MAX][MAX], dist[MAX], used[MAX]={0};
    int i,j,u,min;
    printf("Number of vertices: ");
    scanf("%d",&n);
    printf("Enter adjacency matrix (0 means no edge):\n");
    for(i=0;i<n;i++) for(j=0;j<n;j++) scanf("%d",&a[i][j]);
    printf("Source vertex index (0 to %d): ",n-1);
    scanf("%d",&src);
    for(i=0;i<n;i++) dist[i]=INF;
    dist[src]=0;
    for(i=0;i<n-1;i++) {
        min=INF; u=-1;
        for(j=0;j<n;j++) if(!used[j] && dist[j]<min) {
            min=dist[j]; u=j;
        }
        if(u==-1) break;
        used[u]=1;
        for(j=0;j<n;j++)
            if(!used[j] && a[u][j]>0 && dist[u]+a[u][j]<dist[j])
                dist[j]=dist[u]+a[u][j];
    }
    printf("Vertex\\tDistance from source\n");
    for(i=0;i<n;i++) {
        if(dist[i]==INF) printf("%d\\tUnreachable\n",i);
        else printf("%d\\t%d\n",i,dist[i]);
    }
    return 0;
}
```

Output:

```
Sample run (example graph):
Number of vertices: 5
Enter adjacency matrix:
0 10 0 30 100
10 0 50 0 0
0 50 0 20 10
30 0 20 0 60
100 0 10 60 0
Source vertex index: 0
Vertex  Distance from source
0       0
1       10
2       50
3       30
4       60
```

5. DAG for topological sorting

Topological sorting using depth-first search.

```
/* Topological sorting of a DAG using DFS.
   This example uses the directed graph shown by the adjacency matrix. */
#include <stdio.h>
#define N 6

int graph[N][N] = {
    {0,1,1,0,0,0},
    {0,0,0,1,0,0},
    {0,0,0,1,1,0},
    {0,0,0,0,0,1},
    {0,0,0,0,0,1},
    {0,0,0,0,0,0}
};
int visited[N], stack[N], top=-1;

void dfs(int v) {
    int i;
    visited[v]=1;
    for(i=0;i<N;i++)
        if(graph[v][i] && !visited[i]) dfs(i);
    stack[++top]=v;
}
int main(void) {
    int i;
    for(i=0;i<N;i++) if(!visited[i]) dfs(i);
    printf("Topological order: ");
    while(top>=0) printf("%c ", 'A'+stack[top--]);
    printf("\n");
    return 0;
}
```

Output:

```
Topological order: A C E B D F
```

6. Insert 76, 93, 40, 47, 10 and 55 into a hash table of size 7 using linear probing

Hash function: h(k) = k mod 7.

```
#include <stdio.h>
#define SIZE 7

int main(void) {
    int table[SIZE], keys[]={76,93,40,47,10,55};
    int i,j,index,pos;
    for(i=0;i<SIZE;i++) table[i]=-1;
    for(i=0;i<6;i++) {
        index=keys[i]%SIZE;
        for(j=0;j<SIZE;j++) {
            pos=(index+j)%SIZE;
            if(table[pos]==-1) { table[pos]=keys[i]; break; }
        }
        if(j==SIZE) printf("%d could not be inserted\n",keys[i]);
    }
    printf("Index : Value\n");
    for(i=0;i<SIZE;i++) printf("%d : %d\n",i,table[i]);
    return 0;
}
```

Output:

```
Index : Value
0 : 47
1 : 55
2 : 93
3 : 10
4 : -1
5 : 40
6 : 76
```

7. Insert 22, 30 and 50 using quadratic probing

Table size = 7; h(k) = k mod 7; f(i) = i^2.

```
#include <stdio.h>
#define SIZE 7

int main(void) {
    int table[SIZE], keys[]={22,30,50};
    int i,j,index,pos;
```

```
    for(i=0;i<SIZE;i++) table[i]=-1;
    for(i=0;i<3;i++) {
        index=keys[i]%SIZE;
        for(j=0;j<SIZE;j++) {
            pos=(index+j*j)%SIZE;
            if(table[pos]==-1) { table[pos]=keys[i]; break; }
        }
        if(j==SIZE) printf("%d could not be inserted\n",keys[i]);
    }
    printf("Index : Value\n");
    for(i=0;i<SIZE;i++) printf("%d : %d\n",i,table[i]);
    return 0;
}
```

Output:

```
Index : Value
0 : -1
1 : 22
2 : 30
3 : -1
4 : -1
5 : 50
6 : -1
```

8. Insert 27, 43, 69 and 72 using double hashing

h1(k) = k mod 7; h2(k) = 1 + (k mod 5).

```
#include <stdio.h>
#define SIZE 7

int main(void) {
    int table[SIZE], keys[]={27,43,69,72};
    int i,j,h1,h2,pos;
    for(i=0;i<SIZE;i++) table[i]=-1;
    for(i=0;i<4;i++) {
        h1=keys[i]%SIZE;
        h2=1+(keys[i]%5);
        for(j=0;j<SIZE;j++) {
            pos=(h1+j*h2)%SIZE;
            if(table[pos]==-1) { table[pos]=keys[i]; break; }
        }
        if(j==SIZE) printf("%d could not be inserted\n",keys[i]);
    }
    printf("Index : Value\n");
    for(i=0;i<SIZE;i++) printf("%d : %d\n",i,table[i]);
    return 0;
}
```

Output:

```
Index : Value
0 : -1
1 : 43
2 : 72
3 : -1
4 : 69
5 : -1
6 : 27
```

9. Prove the statement about quadratic probing

Proof: Let the prime table size be m. With quadratic probing, the probe sequence is (h(k) + i²) mod m. For a prime table size, the first (m+1)/2 probe positions are distinct. If the table is at least half empty, there are at least (m+1)/2 empty positions. The probe set and the empty-position set each contain at least (m+1)/2 positions in a table of m positions, so the two sets must intersect. Therefore, at least one probe reaches an empty position and the new element can be inserted. Hence proved.

10. Implement a hash data structure with initialize, find and insert routines using separate chaining

Separate chaining stores colliding keys in a linked list at the same hash-table bucket.

```
#include <stdio.h>
#include <stdlib.h>
#define SIZE 10

struct Node {
    int data;
```

```
    struct Node *next;
};
struct Node *table[SIZE];

void initialize(void) {
    int i;
    for(i=0;i<SIZE;i++) table[i]=NULL;
}
void insert(int key) {
    int index=key%SIZE;
    struct Node *p=(struct Node *)malloc(sizeof(struct Node));
    if(p==NULL) { printf("Memory allocation failed\n"); return; }
    p->data=key;
    p->next=table[index];
    table[index]=p;
    printf("%d inserted into bucket %d\n",key,index);
}
int find(int key) {
    int index=key%SIZE;
    struct Node *p=table[index];
    while(p!=NULL) {
        if(p->data==key) return index;
        p=p->next;
    }
    return -1;
}
int main(void) {
    int pos;
    initialize();
    insert(25);
    insert(35);
    insert(45);
    pos=find(35);
    if(pos!=-1) printf("35 found in bucket %d\n",pos);
    else printf("35 not found\n");
    return 0;
}
```

Output:

```
25 inserted into bucket 5
35 inserted into bucket 5
45 inserted into bucket 5
35 found in bucket 5
```